

Documentation évaluation

Auteur: Widad

March 19, 2026

Abstract

Documentation du travail effectué pour l'évaluation de réponses. Dernier edit: 19/03/2026

1 évaluation des réponses

Pour le moment on fait une évaluation basée sur deux approches:

- évaluation humaine : utile/partiellement utile/inutile. Juge uniquement l'utilité de la réponse.
- évaluation basée sur des critères de performance: rapidité, consommation de ressources, absence d'erreurs.

1.1 automatiser le dataset avec benchmark.py

Ce script prend en entrée un fichier csv de tickets IT tickets_scenario.1.csv avec leur contexte. A l'exécution avec input de l'utilisateur, on obtient une base de tickets évalués avec leurs réponses et métriques: tickets_evalues.json

Modifications faites par rapport au fichier initial de akram:

- on capture la date et heure.
- on stocke aussi le nombre de tokens. Utilise les compteurs natifs de Ollama `prompt_eval_count + eval_count`
- on récupère et on stocke le feedback utilisateur par ticket.
- on peut tester sur différents modèles. Par défaut (pas d'arguments) `phi3:latest` est utilisé. Pour tester avec `mistral:latest` et `phi3:latest` utiliser la commande suivante:

```
src/benchmark/benchmark.py phi3:latest mistral:latest
```

Observation: temps d'exécution très variable... 16 à 95+ secondes pour la même question (plusieurs essais).

1.2 `evaluer_reponses.py`

C'est le script principal qui évalue les réponses aux tickets. Il contient aussi une fonction de comparaison des différents modèles.

A l'exécution du fichier 2 choses se produisent:

- Il génère le fichier `tickets_evalues_scores.json`.
- Il affiche également dans le terminal une comparaison des différents modèles en se basant sur les tickets correspondants.

1.3 `tickets_evalues_fake.json`

Base de tickets fictifs et leurs réponses et métriques également fictives.

Variables retenues:

- `id_ticket`
- `contexte`
- `ticket`
- `modele`
- `dateheure`
- `reponse`
- `feedback_utilisateur`
- `temps_reponse (s)`
- `nombre_tokens`

Les améliorations à faire ici: il faut mettre à jour les noms de modèles si d'autres que ceux-ci sont utilisés + faire un programme qui génère automatiquement ce fichier à partir de vrais (tickets,réponses) + il faut faire le lien avec `evaluation_manager.py` qui récupère les feedbacks utilisateur (ajouter une option "moyennement utile" ici) + également il faut remplacer "ERREUR" par les vrais messages d'erreur qu'on obtient qu'on l'agent se plante (éventuellement définir un tel message si pas encore fait).

1.4 `config_evaluation.py`

Ce fichier est en gros la formule qu'on utilise pour calculer le score qui va être attribué à chaque ticket évalué. Il peut être édité à volonté par exemple pour faire varier le poids attribué à chaque variable. Pour cette même raison c'est un fichier séparé du code d'évaluation.

Pour le moment on part sur 4 variables:

- `erreur_penalty` (si erreur, on pénalise avec un score de 0)

- pertinence (feedback utilisateur: utile, pas utile, moyennement utile)
- vitesse (temps de réponse)
- efficacité (nombre de tokens)

Ensuite on combine ces 4 critères avec des poids différents pour obtenir un score par ticket. J’ai fait le choix (modifiable) de prendre 35% pertinence, 25% pénalité erreur, 20% vitesse, et 20% efficacité. Ainsi la pertinence est le critère le plus important, les erreurs comptent, la vitesse et l’efficacité comptent un peu moins.

Pour calculer les sous_scores pour chaque critère (vitesse, pertinence etc) le feedback textuel est converti en valeurs numériques, et de même les valeurs temporelles (continues) relatives à la vitesse sont converties en scores suivant des seuils (par exemple: moins d’ 1.0s = excellent, plus de 30.0 = lent, etc).

1.5 tickets_evalues_scores.json

Ici on stocke les scores et sous-scores attribués aux tickets.

2 La suite

- On pourra améliorer le volet évaluation humaine en ajoutant d’autres aspects comme le ton de la réponse, l’exactitude de la réponse (par exemple pour une question dont la réponse est déjà connue).
- On peut faire évaluer les réponses par un modèle plus puissant, avec par exemple une grille de critères.
- On peut demander à l’humain ou au modèle de choisir la meilleure parmi deux réponses (générées par deux modèles différents) (par exemple pour définir le meilleur modèle pour un domaine particulier).
- Automatisation du dataset: Il faut remplacer le message d’erreur générique (“ERREUR”, c’est juste un placeholder) par les véritables logs de plantage.
- Pour améliorer les critères d’évaluation: quant au feedback utilisateur, on peut intégrer le ton de la réponse. On peut aussi mesurer l’exactitude factuelle des réponses (uniquement dans les cas où la réponse à la question est déjà connue). A/B testing: piste de réflexion. Implémenter le choix forcé entre deux modèles pour identifier le plus performant, notamment par domaine. Évaluation aveugle: Idée de l’encadrant. Randomiser les modèles présentés aux utilisateurs pour supprimer des biais et voir lesquels sont le mieux notés. On peut faire faire les évaluations ci-dessus à une LLM performante avec une grille de critère ou autres instructions. Analyses croisées: on peut faire abstraction du score final et faire des comparaisons par thématique et par critère (ex: modèle X plus rapide que modèle Y pour thème Z).

- les seuils pour les sous-score (par exemple ce qui constitue une réponse lente ou rapide) ont été choisis de manière un peu aléatoire... À préciser.

3 modifs cette semaine

- automatisation du dataset (benchmark.py run les tickets depuis le fichier de tickets csv, appelle le bon modèle par le biais d'ollama et stocke les reponses et les metriques y compris nb de tokens, modèle, feedback user dans tickets_evalues.json)
- benchmark.py prend une liste de modeles pour pouvoir comparer les reponses des modeles à une meme question, sinon utilise phi3:latest à l'exécution.

4 bibliographie

- [1] <https://blog.jetbrains.com/pycharm/2024/08/how-to-build-chatbots-with-langchain/#step-1-project-creation,-prerequisites,-and-required-library-installation>
- [2] <https://hyperskill.org/courses/149-ai-agents-theory-and-practice>